Departamento de Electrónica, Sistemas e Informática

Big Data

Spring 2026

Examples on Machine Learning: Decision Trees and Random Forest

Profesor: Pablo Camarillo Ramirez

Alumno: Bryan Edgardo Romo Gonzalez

Create SparkSession

In [1]:
```
import findspark
findspark.init()

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("ML: Decision Trees & Random Forest") \
    .master("spark://spark-master:7077") \
    .config("spark.ui.port", "4040") \
    .getOrCreate()
```

```
sc = spark.sparkContext
sc.setLogLevel("INFO")

# Optimization (reduce the number of shuffle partitions)
spark.conf.set("spark.sql.shuffle.partitions", "5")
```

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/24 01:59:50 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java
classes where applicable
```

Decision Trees

Collect Data

In [2]:
```
from pcamarillor.spark_utils import SparkUtils
# Create a small dataset as a list of tuples
# Format: (label, x1, x2)
data = [
      (0, 1.0, 0.5),
      (1, 2.0, 1.5),
      (0, 1.5, 0.2),
      (1, 2.2, 1.0),
      (0, 1.0, -0.3),
      (1, 2.5, 1.0)
]

# Define schema for the DataFrame
schema = SparkUtils.generate_schema([("label", "int"), ("x1", "float"), ("x2", "float")])

# Convert list to a DataFrame
df = spark.createDataFrame(data, schema)
```

In [3]:
```
df.show()
```

```
+-----+---+----+
|label| x1|  x2|
+-----+---+----+
|    0|1.0| 0.5|
|    1|2.0| 1.5|
|    0|1.5| 0.2|
|    1|2.2| 1.0|
|    0|1.0|-0.3|
|    1|2.5| 1.0|
+-----+---+----+
```

Assemble the features into a single vector column

In [4]:
```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(inputCols=["x1", "x2"], outputCol="features")
data_with_features = assembler.transform(df).select("label", "features")
```

In [5]:
```
data_with_features.show()
```

```
[Stage 2:>                                                          (0 + 1) / 1]
+-----+--------------------+
|label|            features|
+-----+--------------------+
|    0|           [1.0,0.5]|
|    1|           [2.0,1.5]|
|    0|[1.5,0.2000000029...|
|    1|[2.20000004768371...|
|    0|[1.0,-0.300000011...|
|    1|           [2.5,1.0]|
+-----+--------------------+
```

Data splitting

80% training data and 20% testing data

In [6]:
```
train_df, test_df = data_with_features.randomSplit([0.8, 0.2], seed=57)
```

Show dataset (for debugging)

In [7]:
```
print("Original Dataset")
df.show()

# Print train dataset
print("train set")
train_df.show()
```

```
Original Dataset
+-----+---+----+
|label| x1|  x2|
+-----+---+----+
|    0|1.0| 0.5|
|    1|2.0| 1.5|
|    0|1.5| 0.2|
|    1|2.2| 1.0|
|    0|1.0|-0.3|
|    1|2.5| 1.0|
+-----+---+----+

train set
+-----+--------------------+
|label|            features|
+-----+--------------------+
|    0|           [1.0,0.5]|
|    0|[1.5,0.2000000029...|
|    1|           [2.0,1.5]|
|    0|[1.0,-0.300000011...|
|    1|           [2.5,1.0]|
+-----+--------------------+
```

Create ML Model

In [8]:
```
from pyspark.ml.classification import DecisionTreeClassifier

# Initialize and train the Decision Tree model
dt = DecisionTreeClassifier(labelCol="label", featuresCol="features")
```

Train ML Model

In [9]:
```
dt_model = dt.fit(train_df)

# Display model summary
print("Decision Tree model summary:{0}".format(dt_model.toDebugString))
```

```
26/04/24 02:00:19 WARN DecisionTreeMetadata: DecisionTree reducing maxBins from 32 to 5 (= number of training instanc
es)
Decision Tree model summary:DecisionTreeClassificationModel: uid=DecisionTreeClassifier_0ff41279dba9, depth=1, numNod
es=3, numClasses=2, numFeatures=2
  If (feature 0 <= 1.75)
   Predict: 0.0
  Else (feature 0 > 1.75)
   Predict: 1.0
```

Persist the model

In [10]:
```
model_path = "/opt/spark/work-dir/data/mlmodels/dt/dt1"
dt_model.write().overwrite().save(model_path)
```

In [11]:
```
!ls -lah /opt/spark/work-dir/data/mlmodels/dt/dt1/data
```

```
total 4.0K
drwxr-xr-x 1 root root  512 Apr 24 02:00 .
drwxr-xr-x 1 root root  512 Apr 24 02:00 ..
-rw-r--r-- 1 root root 3.8K Apr 24 02:00 part-00000-512ef744-3ed7-4330-abde-be89f2751182-c000.snappy.parquet
-rw-r--r-- 1 root root   40 Apr 24 02:00 .part-00000-512ef744-3ed7-4330-abde-be89f2751182-c000.snappy.parquet.crc
-rw-r--r-- 1 root root    0 Apr 24 02:00 _SUCCESS
-rw-r--r-- 1 root root    8 Apr 24 02:00 ._SUCCESS.crc
```

Predictions

In [12]:
```
# Use the trained model to make predictions on the test data
predictions = dt_model.transform(test_df)

# Show predictions
predictions.select("features", "prediction").show()
```

```
+--------------------+----------+
|            features|prediction|
+--------------------+----------+
|[2.20000004768371...|       1.0|
+--------------------+----------+
```

In [13]:
```
from pyspark.ml.classification import DecisionTreeClassificationModel

# Retreive the saved model
saved_dt_model = DecisionTreeClassificationModel.load(model_path)

# Use the trained model to make predictions on the test data
predictions = saved_dt_model.transform(test_df)

# Show predictions
predictions.select("features", "prediction").show(truncate=False)
```

```
+----------------------+----------+
|features              |prediction|
+----------------------+----------+
|[2.200000047683716,1.0]|1.0       |
+----------------------+----------+
```

Test ML Model

In [14]:
```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

evaluator = MulticlassClassificationEvaluator(labelCol="label",
                            predictionCol="prediction")
```

```
accuracy = evaluator.evaluate(predictions,
                  {evaluator.metricName: "accuracy"})
print(f"Accuracy: {accuracy}")

f1 = evaluator.evaluate(predictions,
                {evaluator.metricName: "f1"})
print(f"F1 Score: {f1}")
```

```
Accuracy: 1.0
F1 Score: 1.0
```

Random Forest

In [15]:
```
from pyspark.ml.classification import RandomForestClassifier

# Train the model
rf = RandomForestClassifier(
    labelCol="label",
    featuresCol="features",
    numTrees=3,
    maxDepth=5,
    seed=42
)

rf_model = rf.fit(train_df)

# Save the model
rf_path = "/opt/spark/work-dir/data/mlmodels/rf/rf1"
rf_model.write().overwrite().save(rf_path)
print(f"Random forest model generated: {rf_model.toDebugString}")
```

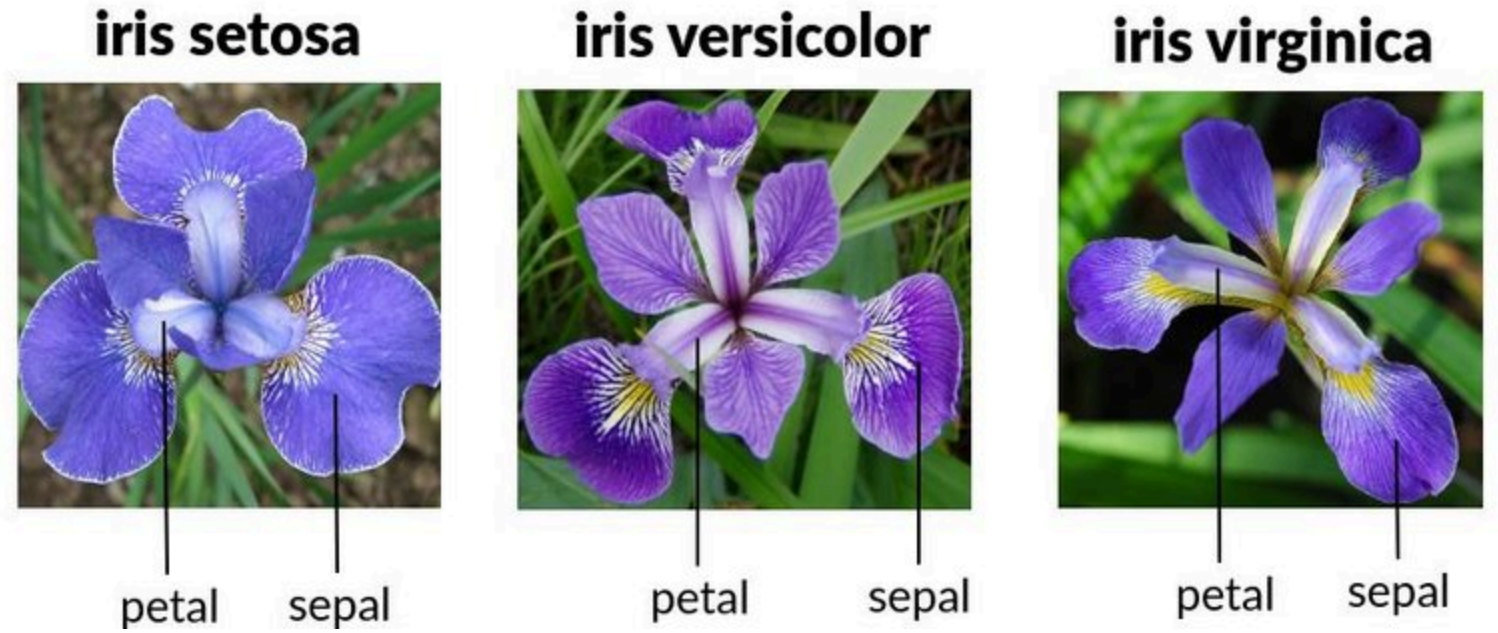
```
26/04/24 02:00:38 WARN DecisionTreeMetadata: DecisionTree reducing maxBins from 32 to 5 (= number of training instanc
es)
```

```
Random forest model generated: RandomForestClassificationModel: uid=RandomForestClassifier_e2a17a3bc115, numTrees=3,
numClasses=2, numFeatures=2
  Tree 0 (weight 1.0):
    Predict: 0.0
  Tree 1 (weight 1.0):
    Predict: 0.0
  Tree 2 (weight 1.0):
    If (feature 0 <= 1.25)
     Predict: 0.0
    Else (feature 0 > 1.25)
     Predict: 1.0
```

In [16]:
```python
from pyspark.ml.classification import RandomForestClassificationModel
# Read the RF model
rf_model_saved = rf_model.load(rf_path)

# Make predictions on test data
predictions_rf = rf_model_saved.transform(test_df)

evaluator = MulticlassClassificationEvaluator(labelCol="label",
                            predictionCol="prediction")

accuracy_rf = evaluator.evaluate(predictions_rf,
                  {evaluator.metricName: "accuracy"})
print(f"Accuracy of Random Forest: {accuracy}")

f1_rf = evaluator.evaluate(predictions_rf,
                {evaluator.metricName: "f1"})
print(f"F1 Score of Random Forest: {f1}")
```

```
Accuracy of Random Forest: 1.0
F1 Score of Random Forest: 1.0
```

Lab 11: Classifying Iris Dataset



Data collection

In [17]:
```
!ls /opt/spark/work-dir/data/ml/decision_trees/
```

```
Iris.csv
```

In [18]:
```
from spark_utils import SparkUtils

# Define schema for the DataFrame

iris_schema = SparkUtils.generate_schema([
    ("Id", "int"),
    ("SepalLengthCm", "float"),
    ("SepalWidthCm", "float"),
    ("PetalLengthCm", "float"),
    ("PetalWidthCm", "float"),
    ("Species", "string")])

# Source: https://raw.githubusercontent.com/selva86/datasets/master/Iris.csv
```

```
iris_df = spark.read \
                .option("header", "true") \
                .schema(iris_schema) \
                .csv("/opt/spark/work-dir/data/ml/decision_trees/")

iris_df.printSchema()
iris_df.show(5)
```

```
root
 |-- Id: integer (nullable = true)
 |-- SepalLengthCm: float (nullable = true)
 |-- SepalWidthCm: float (nullable = true)
 |-- PetalLengthCm: float (nullable = true)
 |-- PetalWidthCm: float (nullable = true)
 |-- Species: string (nullable = true)

+---+-------------+------------+-------------+------------+-----------+
| Id|SepalLengthCm|SepalWidthCm|PetalLengthCm|PetalWidthCm|    Species|
+---+-------------+------------+-------------+------------+-----------+
|  1|          5.1|         3.5|          1.4|         0.2|Iris-setosa|
|  2|          4.9|         3.0|          1.4|         0.2|Iris-setosa|
|  3|          4.7|         3.2|          1.3|         0.2|Iris-setosa|
|  4|          4.6|         3.1|          1.5|         0.2|Iris-setosa|
|  5|          5.0|         3.6|          1.4|         0.2|Iris-setosa|
+---+-------------+------------+-------------+------------+-----------+
only showing top 5 rows
```

In [19]:

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer

feature_cols = ["SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm"]

# Convert string labels (species) to numeric
label_indexer = StringIndexer(inputCol="Species", outputCol="label")

# Assemble features into a single vector column
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")

# Define the Random Forest model
rf = RandomForestClassifier(
    labelCol="label",
    featuresCol="features",
```

```
    numTrees=100,
    maxDepth=5,
    seed=42
)

# Build a pipeline
pipeline = Pipeline(stages=[label_indexer, assembler, rf])

# Split the data into training and test sets
train_df, test_df = iris_df.randomSplit([0.8, 0.2], seed=42)

# Train the model
model = pipeline.fit(train_df)

# Make predictions on test data
predictions = model.transform(test_df)
predictions.show()
```

```
+---+-------------+------------+-------------+------------+--------------+-----+--------------------+--------------
-----+--------------------+----------+
| Id|SepalLengthCm|SepalWidthCm|PetalLengthCm|PetalWidthCm|       Species|label|            features|       rawPredi
ction|         probability|prediction|
+---+-------------+------------+-------------+------------+--------------+-----+--------------------+--------------
-----+--------------------+----------+
|  3|          4.7|         3.2|          1.3|         0.2|   Iris-setosa|  2.0|[4.69999980926513...|     [0.0,0.0,1
00.0]|       [0.0,0.0,1.0]|       2.0|
|  7|          4.6|         3.4|          1.4|         0.3|   Iris-setosa|  2.0|[4.59999990463256...|     [0.0,0.0,1
00.0]|       [0.0,0.0,1.0]|       2.0|
|  9|          4.4|         2.9|          1.4|         0.2|   Iris-setosa|  2.0|[4.40000009536743...|     [0.0,0.0,1
00.0]|       [0.0,0.0,1.0]|       2.0|
| 14|          4.3|         3.0|          1.1|         0.1|   Iris-setosa|  2.0|[4.30000019073486...|     [0.0,0.0,1
00.0]|       [0.0,0.0,1.0]|       2.0|
| 20|          5.1|         3.8|          1.5|         0.3|   Iris-setosa|  2.0|[5.09999990463256...|     [0.0,0.0,1
00.0]|       [0.0,0.0,1.0]|       2.0|
| 24|          5.1|         3.3|          1.7|         0.5|   Iris-setosa|  2.0|[5.09999990463256...|      [3.0,0.0,
97.0]|     [0.03,0.0,0.97]|       2.0|
| 30|          4.7|         3.2|          1.6|         0.2|   Iris-setosa|  2.0|[4.69999980926513...|     [0.0,0.0,1
00.0]|       [0.0,0.0,1.0]|       2.0|
| 36|          5.0|         3.2|          1.2|         0.2|   Iris-setosa|  2.0|[5.0,3.2000000476...|      [2.0,0.0,
98.0]|     [0.02,0.0,0.98]|       2.0|
| 46|          4.8|         3.0|          1.4|         0.3|   Iris-setosa|  2.0|[4.80000019073486...|     [0.0,0.0,1
00.0]|       [0.0,0.0,1.0]|       2.0|
| 47|          5.1|         3.8|          1.6|         0.2|   Iris-setosa|  2.0|[5.09999990463256...|     [0.0,0.0,1
00.0]|       [0.0,0.0,1.0]|       2.0|
| 48|          4.6|         3.2|          1.4|         0.2|   Iris-setosa|  2.0|[4.59999990463256...|     [0.0,0.0,1
00.0]|       [0.0,0.0,1.0]|       2.0|
| 50|          5.0|         3.3|          1.4|         0.2|   Iris-setosa|  2.0|[5.0,3.2999999523...|      [2.0,0.0,
98.0]|     [0.02,0.0,0.98]|       2.0|
| 52|          6.4|         3.2|          4.5|         1.5|Iris-versicolor|  0.0|[6.40000009536743...|[99.05446006387
32...|[0.99054460063873...|       0.0|
| 56|          5.7|         2.8|          4.5|         1.3|Iris-versicolor|  0.0|[5.69999980926513...|[99.19196006387
32...|[0.99191960063873...|       0.0|
| 63|          6.0|         2.2|          4.0|         1.0|Iris-versicolor|  0.0|[6.0,2.2000000476...|[96.47591016782
33...|[0.96475910167823...|       0.0|
| 70|          5.6|         2.5|          3.9|         1.1|Iris-versicolor|  0.0|[5.59999990463256...|[99.36888314079
62...|[0.99368883140796...|       0.0|
| 78|          6.7|         3.0|          5.0|         1.7|Iris-versicolor|  0.0|[6.69999980926513...|      [1.0,99.
0,0.0]|     [0.01,0.99,0.0]|       1.0|
|101|          6.3|         3.3|          6.0|         2.5| Iris-virginica|  1.0|[6.30000019073486...|     [0.0,100.
0,0.0]|       [0.0,1.0,0.0]|       1.0|
```

```
|113|          6.8|          3.0|          5.5|          2.1| Iris-virginica|  1.0|[6.80000019073486...|     [0.0,100.
0,0.0]|       [0.0,1.0,0.0]|       1.0|
|117|          6.5|          3.0|          5.5|          1.8| Iris-virginica|  1.0|[6.5,3.0,5.5,1.79...|     [0.0,100.
0,0.0]|       [0.0,1.0,0.0]|       1.0|
+---+-------------+-------------+-------------+-------------+---------------+-----+--------------------+---------------
-----+--------------------+----------+
only showing top 20 rows
```

Decision Trees

Data Splitting

In [20]:
```
from pyspark.ml.feature import StringIndexer, VectorAssembler

# 1. Convertir la columna de texto "Species" a numérica ("label")
label_indexer = StringIndexer(inputCol="Species", outputCol="label")
iris_df_indexed = label_indexer.fit(iris_df).transform(iris_df)

# 2. Ensamblar las características en un vector como en el código inicial
feature_cols = ["SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm"]
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
data_with_features = assembler.transform(iris_df_indexed).select("label", "features")

# 3. Dividir los datos (80% entrenamiento, 20% prueba)
train_df, test_df = data_with_features.randomSplit([0.8, 0.2], seed=57)

print("train set")
train_df.show(5)
```

```
train set
+-----+--------------------+
|label|            features|
+-----+--------------------+
|  0.0|[4.30000019073486...|
|  0.0|[4.40000009536743...|
|  0.0|[4.40000009536743...|
|  0.0|[4.40000009536743...|
|  0.0|[4.5,2.2999999523...|
+-----+--------------------+
only showing top 5 rows
```

Create ML Model

In [21]:
```
from pyspark.ml.classification import DecisionTreeClassifier

# Inicializar el modelo Decision Tree
dt = DecisionTreeClassifier(labelCol="label", featuresCol="features")
```

Train ML Model

In [22]:
```
# Entrenar el modelo
dt_model = dt.fit(train_df)

# Mostrar el resumen del árbol generado
print("Decision Tree model summary: {0}".format(dt_model.toDebugString))
```

```
Decision Tree model summary: DecisionTreeClassificationModel: uid=DecisionTreeClassifier_4e45c200c1cc, depth=5, numNo
des=15, numClasses=3, numFeatures=4
  If (feature 2 <= 2.449999988079071)
   Predict: 0.0
  Else (feature 2 > 2.449999988079071)
   If (feature 2 <= 4.8500001430511475)
    If (feature 3 <= 1.6500000357627869)
     Predict: 1.0
    Else (feature 3 > 1.6500000357627869)
     If (feature 0 <= 5.950000047683716)
      Predict: 1.0
     Else (feature 0 > 5.950000047683716)
      Predict: 2.0
   Else (feature 2 > 4.8500001430511475)
    If (feature 3 <= 1.75)
     If (feature 2 <= 4.950000047683716)
      Predict: 1.0
     Else (feature 2 > 4.950000047683716)
      If (feature 3 <= 1.550000011920929)
       Predict: 2.0
      Else (feature 3 > 1.550000011920929)
       Predict: 1.0
    Else (feature 3 > 1.75)
     Predict: 2.0
```

Persist ML Model

In [23]:
```
# Guardar el modelo en una nueva ruta específica para este dataset
model_path = "/opt/spark/work-dir/data/mlmodels/dt/dt_iris"
dt_model.write().overwrite().save(model_path)

# Verificar que se generaron los archivos
!ls -lah /opt/spark/work-dir/data/mlmodels/dt/dt_iris/data
```

```
total 8.0K
drwxr-xr-x 1 root root  512 Apr 24 02:02 .
drwxr-xr-x 1 root root  512 Apr 24 02:02 ..
-rw-r--r-- 1 root root 4.2K Apr 24 02:02 part-00000-2b9047ae-419f-4309-92b6-83a005ab261d-c000.snappy.parquet
-rw-r--r-- 1 root root   44 Apr 24 02:02 .part-00000-2b9047ae-419f-4309-92b6-83a005ab261d-c000.snappy.parquet.crc
-rw-r--r-- 1 root root    0 Apr 24 02:02 _SUCCESS
-rw-r--r-- 1 root root    8 Apr 24 02:02 ._SUCCESS.crc
```

Test ML Model

In [24]:

```python
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

# Utilizar el modelo entrenado para hacer predicciones en los datos de prueba
predictions = dt_model.transform(test_df)

# Mostrar algunas predicciones
predictions.select("features", "prediction").show(5)

# Evaluar las métricas de Accuracy y F1 Score
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction")

accuracy = evaluator.evaluate(predictions, {evaluator.metricName: "accuracy"})
print(f"Accuracy: {accuracy}")

f1 = evaluator.evaluate(predictions, {evaluator.metricName: "f1"})
print(f"F1 Score: {f1}")
```

```
+--------------------+----------+
|            features|prediction|
+--------------------+----------+
|[5.0,3.4000000953...|       0.0|
|[5.0,3.5,1.299999...|       0.0|
|[5.09999990463256...|       0.0|
|[5.19999980926513...|       0.0|
|[5.40000009536743...|       0.0|
+--------------------+----------+
only showing top 5 rows
Accuracy: 0.9545454545454546
F1 Score: 0.9545454545454545
```

Compare Decision Trees vs Random Forest

In [25]:
```
# Imprimimos las métricas de Decision Tree.

print("--- Comparación de Modelos (Iris Dataset) ---")
print(f"Decision Tree Accuracy: {accuracy}")
print(f"Decision Tree F1 Score: {f1}")
```

```
--- Comparación de Modelos (Iris Dataset) ---
Decision Tree Accuracy: 0.9545454545454546
Decision Tree F1 Score: 0.9545454545454545
```

In [26]:
```
sc.stop()
```

In [ ]:
```

```